

MemoryForge

Associative Memory & Fast Weights

DataForge 2026 / PS-1 - Pathway / Team BitWise



What changes when a model learns an association?

Teach a memory that a handwritten 1 means ALPHA, a 4 means GAMMA and an 8 means BETA. Then query different images of those digits. MemoryForge makes the resulting predictions inspectable: the memory changes, while every encoder parameter stays identical. This is a functional CPU prototype for studying temporary associations, not a claim that new visual categories appear without prior training. [5]

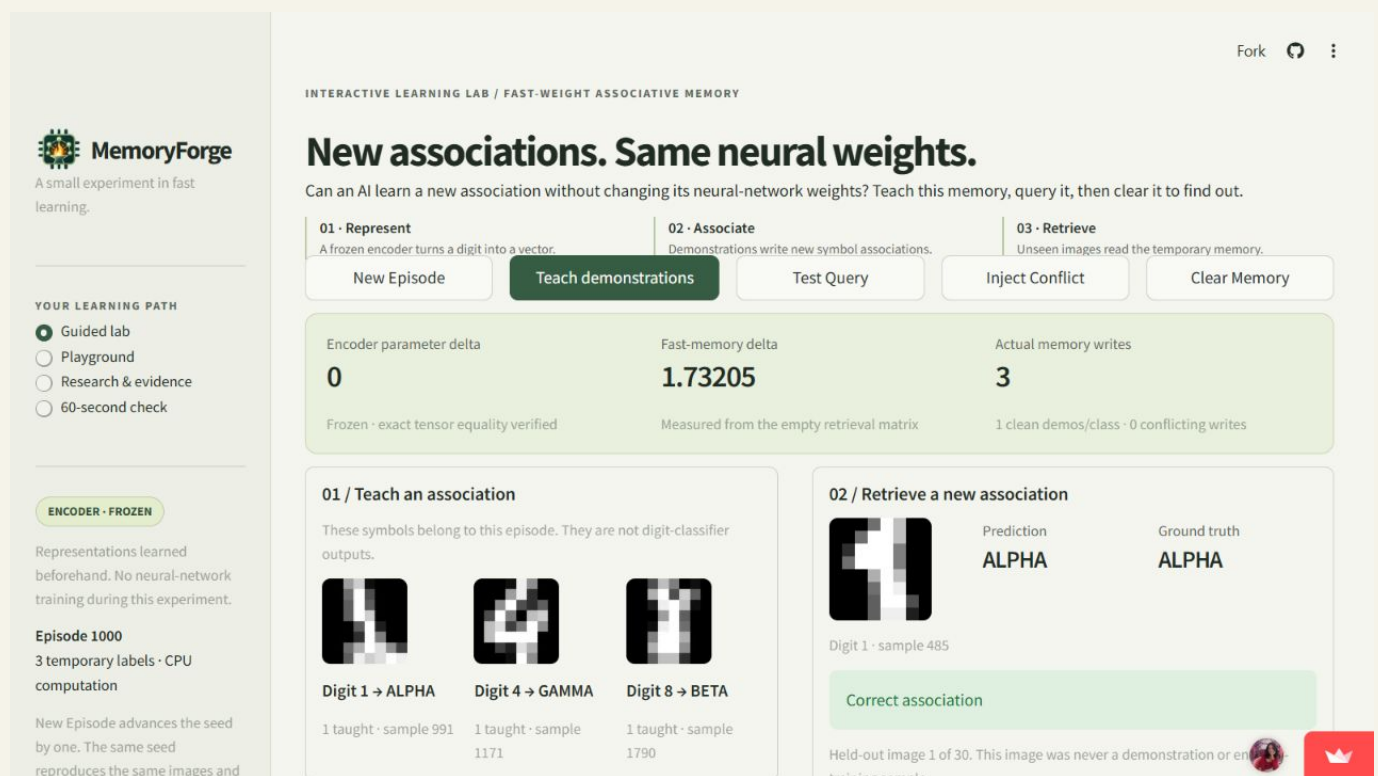


Figure 1. The opening experiment. The public seed-1000 preset has three writes. Encoder parameter delta is 0; fast-memory delta is 1.73205 relative to empty memory. Prediction and truth appear side by side for a query excluded from both demonstrations and encoder training. The values describe this recorded state. [5]

Why make the state visible?

A changing answer does not reveal whether an application retrained a network, stored examples or updated a temporary association. MemoryForge lets a learner examine the write, score an unseen image and erase the learned names. The intended benefit is a causal explanation a learner can check numerically. Improved learning outcomes have not been measured.

The public Streamlit prototype opened without owner sign-in during the 8 September 2026 checks. It includes a guided lab, a shot-count playground, sourced research and a short concept check. The code, checkpoint and evidence are public. PS-1 / Pathway alignment comes from a real computation, one narrow claim and an explicit explanation of the relevant research. [5]

Recognition and association use different state

The encoder has already learned handwriting representations. Digits from scikit-learn supply 1,797 images, each with 64 pixel features scaled by 16. A fixed stratified split assigns 1,347 rows to training and 450 to held-out evaluation. The encoder has linear layers of sizes 64-32-16 with ReLU between them. A ten-class head provides the training objective. [5]

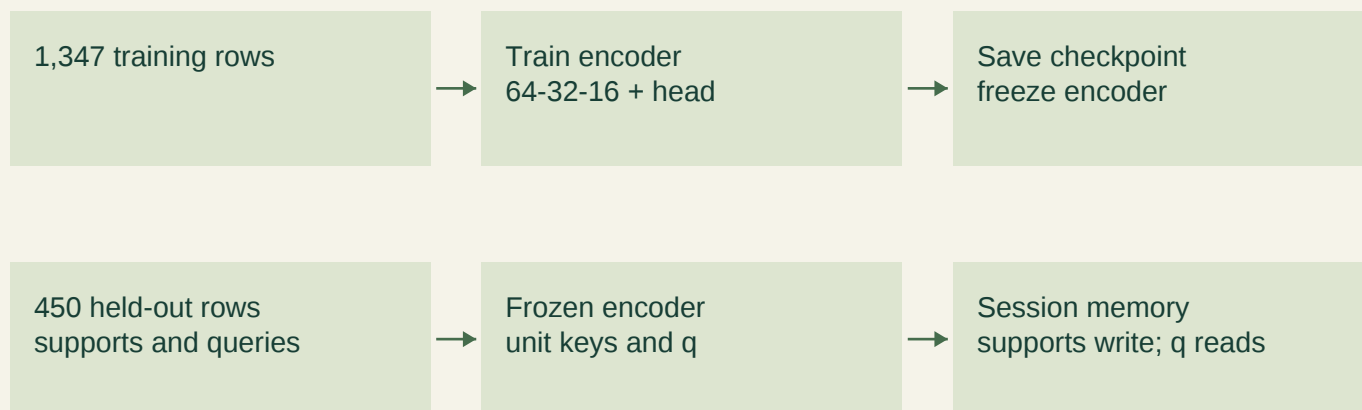


Figure 2. Implemented data flow. Offline training creates the checkpoint. At runtime, frozen embeddings serve distinct held-out supports and queries. Only support keys and their written labels enter memory updates. Query truth evaluates predictions; it never writes memory. The full data-flow document includes initialization, audits and store lifetimes. [5]

Training runs 100 fixed epochs of Adam on CPU. The saved head scores 437/450 on ordinary digit classification, a sanity check distinct from temporary-symbol retrieval. The deployed endpoint loads and validates the checkpoint; it does not train automatically. Shared resources cache frozen embeddings. Each browser owns its temporary LabSession. [5]

The exact implemented rule

For unit embedding k and one-hot symbol v , S accumulates keys and c counts writes. Another unit embedding q queries the class means:

$$\begin{aligned}
 S &\leftarrow S + v k^T \\
 c &\leftarrow c + v \\
 M[i] &= S[i]/c[i] \text{ if } c[i]>0, \text{ else } 0 \\
 \text{scores} &= Mq
 \end{aligned}$$

The outer product selects a row of S . Averaging prevents a label from winning merely because its sum contains more demonstrations. The means are not renormalized. Scores are average key-query dot products, not calibrated confidence. Only written labels can win; empty memory abstains. Memory uses detached float64 CPU tensors, while encoder inference uses float32 and no gradients. [5]

A wrong label is a real memory write

Teach demonstrations adds the next support for each class. Test Query reads another fixed held-out image without changing the memory. Inject Conflict appends an existing support key under the next cyclic symbol. In seed 1000, it writes a digit-1 key as BETA while the true association remains ALPHA. [5]

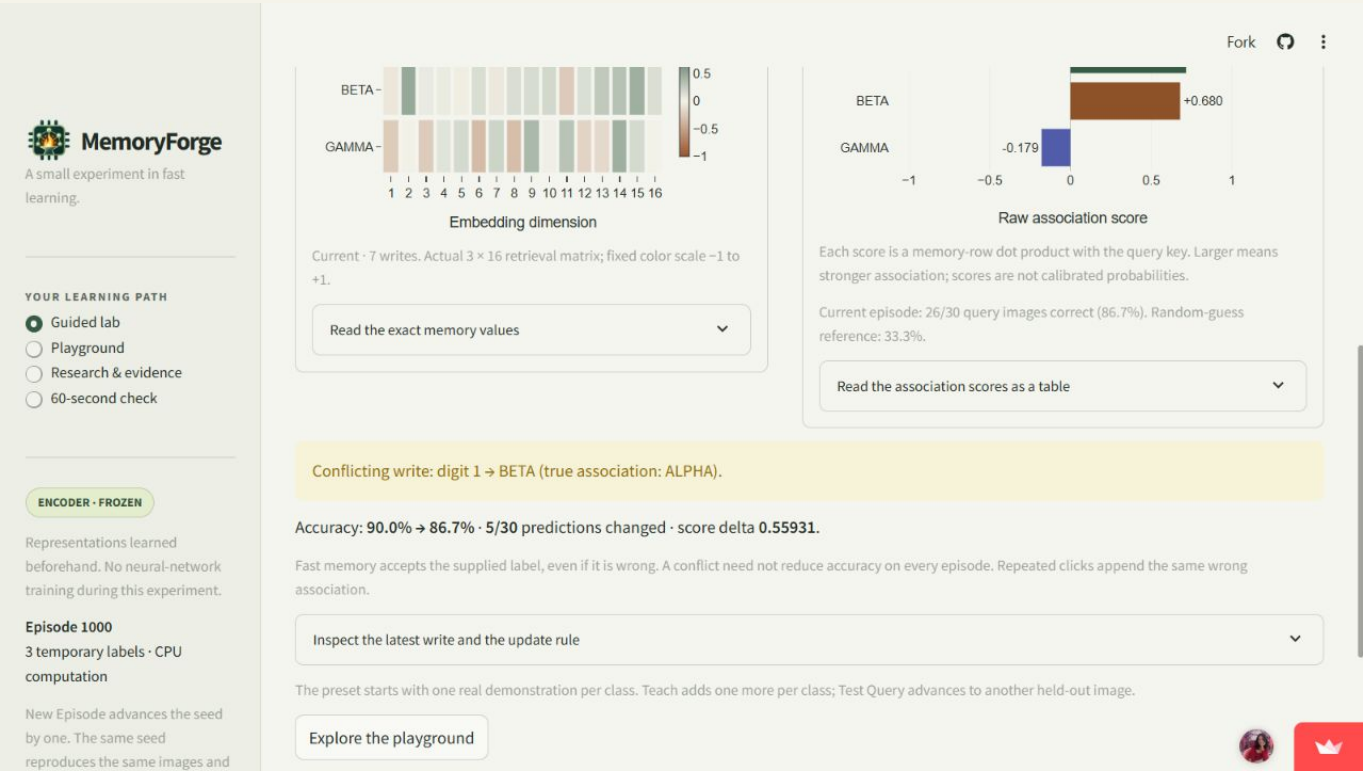


Figure 3. An observed failure. After teaching two supports per class, one contradictory write changes five predictions and reduces correct queries from 27 to 26. The warning identifies the written symbol and unchanged truth; the output reflects computation, not a scripted animation. [5]

Seed 1000 state	Writes	Correct / 30
One support per class	3	23
Two supports per class	6	27
One wrong-label write	7	26
Clear Memory	0	0 (abstention)

Five changed predictions do not mean five newly incorrect answers: the net accuracy loss is one. The wrong key shifts BETA's class mean, which can change the ordering of similarity scores. A conflict can help, hurt or leave another episode unchanged. Clear Memory zeros sums and counts, restoring abstention without altering the encoder. [5]

The displayed fast-memory delta measures the Frobenius distance from the empty retrieval matrix. It can shrink as demonstrations are averaged: the six-write and seven-write states show 1.63242 and 1.59232. A norm is not a measure of knowledge. Exact tensor equality separately checks the encoder's unchanged parameters. [5]

One preset is not the evidence base

The paired suite evaluates seeds 1000-1049. Within each seed, the mapping, 30 query images and nested support pool stay fixed. At 1, 2, 5 and 10 supports per class, it records clean memory and one or three appended wrong-label writes. A zero-shot condition records abstention. [5]

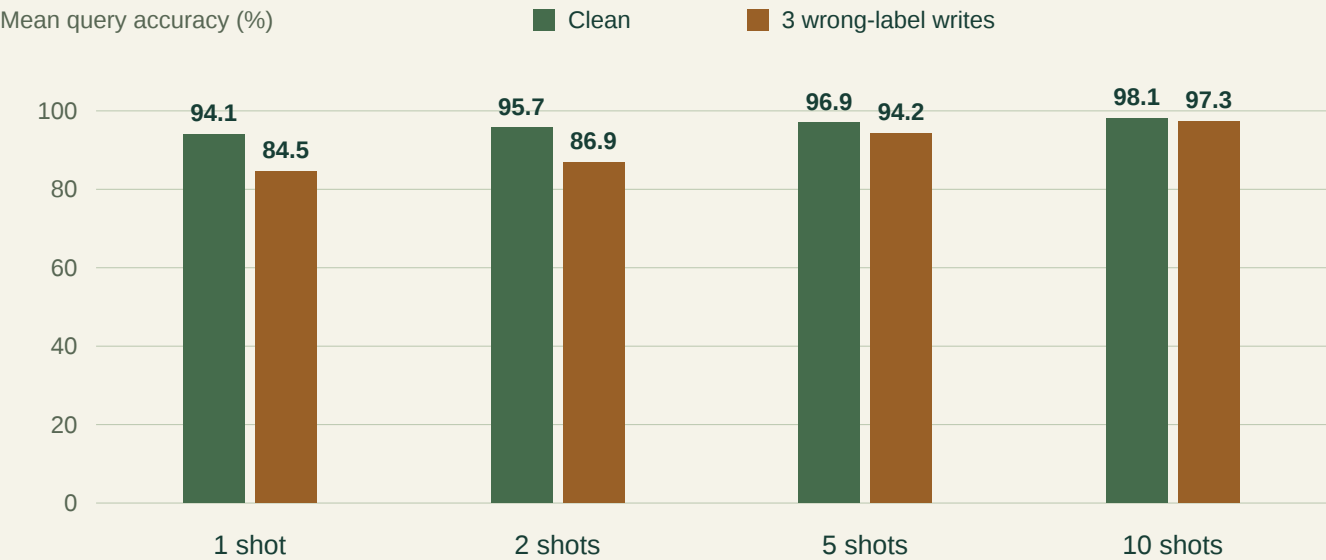


Figure 4. Developer-produced measurements. Mean accuracy across 50 seeds for clean memory and three appended wrong-label writes. The table supplies population standard deviations in percentage points. These describe the fixed episodes, not confidence intervals. The chart comes directly from the saved JSON. [5]

Supports / class	Clean mean +/- SD	3 wrong writes: mean +/- SD
1	94.07% +/- 6.37 pp	84.47% +/- 11.50 pp
2	95.67% +/- 5.26 pp	86.87% +/- 11.34 pp
5	96.93% +/- 4.51 pp	94.20% +/- 7.42 pp
10	98.07% +/- 2.67 pp	97.27% +/- 3.57 pp

The 650 conditions contain 19,500 query outcomes, with reused images rather than independent trials. Clean averages improve over these shot counts, but seed 1000 falls from 90.0% at two clean shots to 73.3% at five. Three wrong writes also represent different noise fractions at different shot counts. The narrowing average accuracy gap is therefore not a controlled fixed-noise-rate comparison. [5]

Encoder delta stays zero throughout, and the saved suite replays exactly in the recorded environment. The final project verification passed 124 tests, with checkpoint regeneration and numerical replay. These checks support reproducibility and implementation correctness. They do not establish external replication, learner effectiveness or production capacity. [5]

Where the research connection holds

BDH separates fixed graph connections from a Hebbian synaptic state. That distinction motivates asking which part of a learning system changes during inference. MemoryForge isolates the idea with a class-mean matrix; it has no BDH graph dynamics, sequence model or biological validation. [1]

BDH-CQ describes demonstration-conditioned memory followed by iterative computation in a separate latent query workspace. Its report leaves exact updates and dimensions proprietary. MemoryForge reads M_q once per query and implements no CQ workspace. It claims neither CQ reasoning performance nor architectural equivalence. [2]

Alternatives expose different trade-offs

DeltaNet uses retrieval-error correction instead of only accumulating associations. Its authors report 100% on MAD In-Context Recall and 52.8% on Memorize, against Transformer baselines of 94.1% and 85.2%. The baseline scores come from earlier work. This synthetic, developer-reported benchmark illustrates task-specific strengths and weaknesses; it is not a deployment or an independent reproduction. [3]

Titans optimizes neural memory at test time with momentum and forgetting. Its memory-module parameters can change during inference. MemoryForge runs no adaptation-time optimizer, so a frozen encoder cannot be used to imply that every test-time-learning architecture freezes all parameters. [4]

Current functionality and its boundaries

The Python 3.12 stack uses PyTorch for the encoder and memory, scikit-learn for bundled digits, NumPy for data handling, and Streamlit with Plotly for controls and charts. Guided lab exposes actual matrices and scores. Playground rebuilds 0-10 clean shots per class while preserving the query set. Research & evidence separates saved evaluations from the live episode. The 60-second check gives explanatory feedback. No model API key is required. [5]

The dominant limitation is the representation: the held-out images belong to digit categories seen during encoder training. Novel categories, language, long-term consolidation and learned forgetting remain untested. Repeated corruption of one key is a specific intervention, not a comprehensive robustness study. Session memory disappears on reload, and no concurrent-load study or human learning evaluation has been performed. [5]

Reuse, future work and sustainability

The present benefit is inspectability: educators can trace a support write, examine a retrieval score and reproduce a conflict with a small CPU lab. A dense memory with C labels and d embedding features stores Cd sums and C counts. The current 3-by-16 memory therefore uses 408 raw bytes for S+c. The full application's memory is larger because it also holds the encoder, embeddings, derived matrices, snapshots and history. This arithmetic is not a scale or throughput benchmark. [5]

A useful next study would keep the corruption fraction fixed while varying memory size and update rule. The existing roadmap also proposes new datasets and broader memory comparisons. Those remain proposals. A learner study should test whether users can distinguish changing state from retraining; a load study should precede classroom-scale hosting.

One possible sustainability model is to keep the educational core open and test paid workshops or managed classroom hosting around it. Revenue would depend on demand, instruction quality and support costs. No current revenue, customers or commercial partnership is claimed. These are planning options, not implemented services.

Run the verified prototype

Use Python 3.12 and the README's isolated-environment setup. Then install requirements and launch the supplied checkpoint:

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
python scripts/evaluate_suite.py
```

Live application: <https://memoryforge.streamlit.app/> Public source: <https://github.com/deeps2710/MemoryForge>

Primary references and continuation

[1] Kosowski et al. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain* (2025 preprint), sections 1.2, 2.2. arXiv:2509.26507v1.

[2] Engdahl et al. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning* (2026 report), sections 3.2-3.3. arXiv:2608.09888v1.

[3] Yang et al. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length* (NeurIPS 2024), sections 2.1-2.2 and Table 1. arXiv:2406.06484v3.

[4] Behrouz et al. *Titans: Learning to Memorize at Test Time* (first submitted 2024-12-31), section 3. arXiv:2501.00663v1.

[5] MemoryForge revision 3751264: src/fast_memory.py, src/lab.py, phase2_demo.json, phase3_evidence.json, phase4_deployment.json and TECHNICAL_WALKTHROUGH.md. Repository links provide the full data and reproduction steps.

Team BitWise: Ved Patel and Deepshikha Rani. Codex assisted implementation, writing and verification. The team remains responsible for technical understanding. Original code, documentation and weights use scoped MIT; the logo, dataset and third-party materials have separate notices in docs/PROVENANCE.md.